CS-E4960 - Software Testing and Quality Assurance D, Lecture, 3.9.2024-26.11.2024

Exploratory Testing

To do: Make a submission **To do:** Receive a grade

Opened: Tuesday, 1 October 2024, 10:00 AM
Due: Tuesday, 22 October 2024, 10:00 AM

Introduction

Exploratory testing is a systematic approach to manual testing that does not rely on strictly scripted test cases. The purpose of this task is to learn about two approaches to exploratory testing through applying them, and to form an advanced understand ing of exploratory testing, including how it can be planned and performed, what choices the tester faces, and how test results can be recorded and reported.

In this task, you must:

- Perform and report on a small exploratory test using **tour-based test management**.
- Perform and report on a small exploratory test using **session-based exploratory testing**.
- Make a **comparative analysis** of the two approaches.

System Under Test (SUT)

The system under test for this assignment is the Oscar e-commerce platform's storefront interface. This interface simulates an e-commerce platform, providing a realistic environment for testing online shopping functionalities. You need to install the Oscar Store following the **Oscar Store Sandbox Installation Guide**. If you don't have it running locally, you can install and run it on your machine. You can either use the local version or the Docker version for your setup.

Installation Instructions for Oscar Store Sandbox

You can follow the easy steps to install the Oscar Store sandbox locally by visiting the official documentation: https://django-oscar.readthedocs.io/en/latest/internals/sandbox.html

- **Local Version:** The sandbox can be set up using a Python virtual environment. This will allow you to interact with Oscar locally, and explore its features in a development environment.
- **Docker Version:** If you prefer to run Oscar using Docker, there are straightforward Docker commands provided to quickly get a sandbox instance in a containerized environment.

You can access the Oscar storefront at http://localhost:8000 once the sandbox is running with `docker run -p 8000:8080/tcp oscarcommerce/django-oscar-sandbox:latest` command. Keep in mind that this command is slightly different from the command on the website, which runs the container on port 8080.

Warning for Local Version: If you are using the local version of the Oscar sandbox, you may see that the Django Debug Toolbar is enabled by default on the website. This toolbar might hinder your acceptance tests because it can cover some of the elements on the page. To disable the toolbar, you can open the `settings.py` file located in the sandbox folder inside the repository. Then, set the `INTERNAL_IPS` list to an empty list like this: `INTERNAL_IPS = []`.

Please read the instructions below carefully from start to finish before you begin this assignment.

Step 1: Tour-based Exploratory Testing

1. Get familiar with tour-based exploratory testing through the lecture materials and by skimming Chapter 4 of Whittaker (2010).
2. Prepare for the test tour:
   - Copy the provided template into a text editor (`tour-report-template.txt`).
   - Choose either "The Landmark Tour" or "The Supermodel Tour" from the book. Fill in the tour name and write a very short summary of the purpose of the tour.
   - Elaborate on the intent of the tour. Fill in the intent of your exploration. The chosen tour and intent should be related to the Oscar storefront's shopping basket – however, you can take any viewpoint that you can justify and that is possible to carry ou t on the Oscar storefront.
3. Take the tour (perform the exploratory test session according to the tour idea and intent).
   - Use a timer to record how long the tour lasts.
   - As you explore, keep notes in the sheet. Remember that the notes should be detailed enough for someone else to understand what you did – what general path you took on the tour.
   - The notes should also address the tour intent: it should be possible to understand how the exploration fulfilled the intent.
   - Also record any faults and issues you find. Faults refer to actual software code faults while issues highlight more general problematic or risky behaviour in the SUT.
4. Once the tour is finished, finalise the report file and save it as a text file. The tour report file must be named `shoppingbasket-tour-<yourname>.txt`.

Step 2: Session-based Test Management

1. Get familiar with session-based test management through the lecture materials and the paper by Bach (2000).
2. Prepare for the test session by copying the provided session sheet into a text editor (`session-sheet.txt`).
3. Perform the exploratory test session.
   - The test is time-boxed to **45 minutes**. Use a timer to make sure you do not exceed this time.
   - As you explore, keep notes in the sheet. Remember that the notes should be detailed enough for someone else to understand what you did – what general path you took on the tour.
   - The notes should also address the session charter: it should be possible to understand how the exploration fulfilled the charter.
   - Also record any faults and issues you find. Faults refer to actual software code faults while issues highlight more general problematic behaviour.
4. Once the test session is over, finalise the session sheet and save it. The session sheet must be named `shoppingbasket-session-sheet-<yourname>.txt`.

Step 3: Analyse, Reflect, and Compare

Look at your testing notes from the tour-based and session-based testing runs and reflect on what happened. You can think of the reflection as your explanation of the test runs during a debriefing with a test lead or scrum master. Fill in the given table with numbers and submit the file. Also, write your reflections using the text box in this assignment. Summarize your reflections so that it compares the results in terms of what was achieved, obstacles/problems that you experienced and why did you receive differen t results if you did.

The comparison does not have to give a completely exhaustive answer to the questions. It should rather demonstrate your thinking and understanding of the themes raised in the question. Focus on essential things that demonstrate what you learned about the two approaches. The answer should be around 300 words.

Tips

- In the test notes, avoid disconnected glimpses and significant gaps in the general path description. Although the test notes do not have to be a precise recording of each action, it should be possible to discern what happened during the exploration a nd what choices you made. Some important details are, e.g., inputs, functions used, views or pages visited, and corresponding outputs, events, and behaviour of the system.
- Write as if the test notes will be compared to someone else's exploration of the same session or tour. There should be enough detail to make such a comparison possible.
- The purpose of this assignment is to get familiar with and understand different approaches to exploratory testing. You will not necessarily find any bugs or significant issues with the SUT. You can still record potential issues although you may not have strong evidence that they can occur.
- It is likely that you will have a memory effect when you do the second step, since you have already performed an exploratory test on the same SUT. This is unavoidable but also illustrates how exploratory testing contributes to learning. You can consider this in your answer if you wish, but it is not mandatory.
- Use what you have learned about test case design in the course: think about how you choose input values, sequences to explore, and other things you could try to reveal defects.
- You can sign in to the storefront with the following credentials:
  - Username: superuser
  - Email: superuser@example.com
  - Password: testing
- If needed in your chosen approach, you can use the developer tools of your browser to inspect the client-side code and what happens behind the scenes in the browser. For example, in Chrome, you can right-click and select Inspect.

Checklist
File shoppingbasket-tour-<yourname>.txt has the tour-based exploratory test report:

- Tour name is filled in.
- Tour summary is filled in.
- Intent is described.
- Start timestamp is filled in.
- Tester name is filled in.
- Test execution time is filled in (minutes or hours:minutes).
- Test notes describe the actions taken in sufficient detail.
- Any found faults are listed. If no faults, write None.
- Any found issues are listed. If no issues, write None.

File shoppingbasket-session-sheet-<yourname>.txt has the session-based test management report:

- Start timestamp is filled in.
- Tester name is filled in.
- Duration is 45 minutes.
- Data files is "#N/A".
- Test notes describe the actions taken in sufficient detail.
- Any found faults are listed. If no faults, write None.
- Any found issues are listed. If no issues, write None.

File analysis-<yourname>.xlsx has your comparison:

- All slots are filled in.
- The analysis is written in the textbox that demonstrates your learning and your reflection on the topic.

Submission: File Submission. Upload each of the three files (session sheet, tour report, and analysis spreadsheet) in your submission.

Grading: The assignment is graded based on 1) the clarity, completeness, and selection of relevant detail in the two test reports, 2) how clearly the testing strategy is explained in the two test reports (it should be possible to follow what was done, while the report is kept brief and to the point), and 3) the insights reported in the analysis comparing the two approaches.

Use of AI and Collaboration with Other Students: Using AI tools to produce an answer for this assignment is not permitted. All parts of the submission must be produced by yourself (apart from files provided in the assignment). Collaboration with other students is not permitted. This is an individual assignment that you must author independently.

- analysis-template.xlsx 30 September 2024, 4:58 PM
- session-sheet-template.txt 30 September 2024, 4:58 PM
- tour-report-template.txt 30 September 2024, 4:58 PM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	Assignment is overdue by: 141 days 4 hours
Last modified	-

Previous activity
◄ Acceptance testing 3: Oscar

Next activity
Static Code Analysis 1: Pylint ►